

SLOTIFY: A SMART APPOINTMENT BOOKING SYSTEM FOR SERVICE-BASED BUSINESSES

A Project Report

Submitted By

AKSH THAKKAR 2403031057002

OM MAURYA 2403031057048

DAKSH PATEL 2403031057065

JAINIL PATEL 2403031057071

in Partial Fulfilment for the Award of

the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Prof. Pratik Gite

Assistant Professor, Computer Engineering Dept., PIET



VADODARA

April - 2026



PARUL UNIVERSITY

CERTIFICATE

This is to Certify that Project - II of 7th Semester entitled “Slotify: A Smart Appointment Booking System for Service-Based Businesses” of Group No. CSE_182 has been successfully completed by

- AKSH THAKKAR – 2403031057002
- OM MAURYA – 2403031057048
- DAKSH PATEL – 2403031057065
- JAINIL PATEL – 2403031057071

under my guidance in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science and Engineering of Parul University in Academic Year 2026-2027.

Date of Submission : _____

Prof. Pratik Gite

Prof. Namrata Patel

Dr. Shailendra K Mishra

Project Guide

Project Coordinator

Head of Department

Computer Engineering Dept., PIET

CSE, PIET

CSE, PIET

Parul University

Parul University

Parul University

Acknowledgements

“The single greatest cause of happiness is gratitude.”

-Auliq-Ice

We express our sincere gratitude to all those who supported and guided us throughout the successful completion of this project. First and foremost, we are thankful to the Almighty for granting us the strength, wisdom, and determination to carry out this work.

We extend our heartfelt thanks to our project guide, **Prof. Pratik Gite**, for his invaluable guidance, continuous encouragement, and constructive feedback at every stage of this project. His expertise and dedication have been a constant source of inspiration. We are also thankful to the faculty members and staff of the Department of Computer Science and Engineering, Parul Institute of Engineering and Technology, Parul University, for providing us with the necessary resources, infrastructure, and academic support.

We sincerely thank **Dr. Shailendra K. Mishra**, Head of Department, CSE, PIET, Parul University, for his support and leadership. Finally, we are deeply grateful to our parents, family members, and well-wishers for their constant moral support and motivation throughout this journey.

AKSH THAKKAR

OM MAURYA

DAKSH PATEL

JAINIL PATEL

CSE, PIET

Parul University, Vadodara

Abstract

Slotify is a smart appointment booking platform developed to simplify scheduling and improve service efficiency for modern businesses. The system provides a digital environment where customers can discover service providers, check real-time availability, book appointments, and receive automated confirmations. It is designed specifically for service-based businesses such as salons, clinics, and wellness centers where scheduling accuracy, transparency, and customer convenience are critical.

The project focuses on solving common issues such as double booking, missed appointments, manual scheduling conflicts, and poor customer engagement. Slotify integrates a dynamic slot generation mechanism, role-based access control, and streamlined booking workflows to improve user experience for customers, staff, and administrators. It also supports features such as booking confirmation, rescheduling, review management, and business-specific service handling.

By combining a web-based frontend with a robust backend and database-driven logic, Slotify offers an efficient and scalable solution for appointment management. The project demonstrates how modern technologies can be used to create a reliable and user-friendly scheduling system that reduces operational overhead while increasing trust and satisfaction for both businesses and customers.

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Tables	viii
List of Figures	ix
1 Introduction	1
1.1 Executive Summary & Platform Capabilities	1
2 Literature Review & Technology Stack	3
2.1 Technology Stack Matrix	3
2.1.1 Frontend Layer	3
2.1.2 Backend Layer	4
3 Software Requirements Specification	5
3.1 End-to-End User Journeys & Workflows	5
3.1.1 Journey 1: Customer Online Booking	5
3.1.2 Journey 2: Specialist Service Execution	6
3.1.3 Journey 3: Verified Review Scorecard	6
3.1.4 Journey 4: Front-Desk Walk-In (POS Mode)	6
3.2 Complete REST API Specification	7

4	System Design	8
4.1	End-to-End System Architecture	8
4.2	Database Schema & Data Model Deep Dive	9
4.2.1	User Model	9
4.2.2	Business Model	9
4.2.3	Service & Staff Models	9
4.2.4	Appointment Model	9
4.2.5	Crucial Unique Concurrency Index	9
5	Core Logic & Mechanics	10
5.1	How the Database Works with the Backend	10
5.1.1	Connection Pooling & Event Lifecycle	10
5.1.2	The Concurrency Problem & The Hardware-Enforced Solution	10
5.1.3	Immutable Snapshot Pricing	11
5.2	Dynamic Slot Engine & Concurrency Control	11
5.2.1	Slot States Explained	11
5.2.2	Dynamic Slot Calculation Algorithm	11
5.3	Email Engine, iCalendar & Notification Systems	11
6	Implementation and Configuration	13
6.1	Backend File-by-File Technical Breakdown	13
6.2	Frontend Architecture & State Management	13
6.2.1	Authentication & Token Refresh Flow	13
6.3	Security, Performance & Error Handling Matrix	14
6.4	Local Setup, Environment Variables & Deployment	14
7	Conclusion	15
7.1	Academic Citation & Institutional Affiliations	15

7.1.1	Research Publication	15
7.1.2	Project Team & Authors	15
8	Future Work	17

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Executive Summary & Platform Capabilities

Appointment scheduling across modern service businesses (salons, clinics, consulting firms, fitness studios) suffers from severe architectural flaws:

1. **Race Conditions & Double Bookings:** When multiple users book the same specialist at the same time, naive database queries fail to prevent overlaps.
2. **Retroactive Price Creep:** Altering a service price alters historical revenue and past booking receipts retroactively if pricing is not snapshot-isolated.
3. **Calendar Isolation:** Lack of 2-way calendar sync causes forgotten appointments, customer no-shows, and fragmented schedules.
4. **Unverified Reviews & Spam:** Platforms allow anyone to post reviews, distorting business credibility.

Slotify resolves all four challenges through:

- **Hardware-Enforced Atomic Concurrency Locks:** Leveraging MongoDB compound partial unique indexing to make double-booking physically impossible at the database engine level.
- **Immutable Snapshot Pricing:** Every booking permanently freezes `priceAtBooking` (Rs. INR) and `durationAtBooking`, preserving audit integrity regardless of future rate card edits.
- **2-Way Calendar Synchronization:** Live RFC-5545 iCalendar (.ics) generation for Apple/Outlook calendars and instant 1-click Google Calendar integration.
- **Brevo Transactional Automation:** Multi-role HTML email pipelines for confirmation, 24-hour reminders, 2-hour reminders, cancellation, rescheduling, and post-session review invitations.

- **Verified Customer Scorecards:** Only customers who completed a verified appointment can submit 1–5 star reviews and feedback for both the business and individual specialists.
- **Multi-Tenant Architecture with Walk-In Mode:** Isolated portals for Customers, Business Owners (Admins), and Staff Specialists, complemented by a rapid Walk-In (POS) interface for reception desks.

Chapter 2

Literature Review & Technology Stack

2.1 Technology Stack Matrix

2.1.1 Frontend Layer

- **React** ($\hat{18}.2.0$): Core UI library providing virtual DOM reconciliation, hooks, and context.
- **Vite** ($\hat{7}.3.0$): Next-generation frontend build tooling with near-instant hot module replacement (HMR).
- **Tailwind CSS** ($\hat{3}.4.0$): Utility-first CSS engine implementing the bespoke Neo-Bauhaus high-contrast design system.
- **React Router DOM** ($\hat{7}.18.2$): Declarative client-side routing, protected route wrappers, and dynamic query param parsing.
- **Axios** ($\hat{1}.19.0$): Promise-based HTTP client equipped with request authorization and response token-refresh interceptors.
- **Lucide React** ($\hat{0}.298.0$): Comprehensive, consistent vector icon library.
- **date-fns** ($\hat{3}.0.6$): Lightweight, immutable date arithmetic, timezone formatting, and calendar math.
- **Lenis** ($\hat{1}.3.26$): Physics-driven smooth inertial scrolling engine.
- **React Hot Toast** ($\hat{2}.4.1$): Non-blocking, accessible toast alerts customized with Neo-Bauhaus borders.
- **React Hook Form** ($\hat{7}.49.2$): High-performance form state management with minimal re-renders.

2.1.2 Backend Layer

- **Node.js** ($\geq 18.0.0$): Scalable JavaScript runtime environment running on V8.
- **Express.js** ($\hat{4}.18.2$): Minimalist web application framework for routing, middleware pipelines, and API endpoints.
- **Mongoose** ($\hat{8}.0.3$): Object Data Modeling (ODM) layer managing schemas, compound indexes, and hooks.
- **Passport.js** ($\hat{0}.7.0$): Authentication middleware orchestrating OAuth 2.0 social login.
- **passport-google-oauth20** ($\hat{2}.0.0$): Strategy for authenticating with Google via OAuth 2.0.
- **jsonwebtoken (JWT)** ($\hat{9}.0.2$): Signed bearer access tokens (short-lived) and refresh tokens (long-lived).
- **bcryptjs** ($\hat{2}.4.3$): Salted 10-round one-way password hashing.
- **node-cron** ($\hat{4}.6.0$): Background task runner executing hourly reminder schedules.
- **@getbrevo/brevo** ($\hat{6}.0.3$): Direct REST API client for transactional HTML emails and iCal attachments.
- **cloudinary** ($\hat{2}.8.0$): Cloud-based media storage and image optimization CDN for business logos and banners.
- **multer** ($\hat{1}.4.5-1ts.1$): In-memory multipart/form-data handler for file uploads.
- **cookie-parser** ($\hat{1}.4.6$): HTTP cookie serialization and parsing for secure refresh token handling.
- **cors** ($\hat{2}.8.5$): Cross-Origin Resource Sharing control with dynamic origin matching.
- **dotenv** ($\hat{16}.3.1$): Environment variable injector.

Chapter 3

Software Requirements Specification

3.1 End-to-End User Journeys & Workflows

3.1.1 Journey 1: Customer Online Booking

1. **Discovery:** Customer navigates to `/search` or `/business/:slug`.
2. **Service Selection:** Customer clicks "Book" on "Haircut & Styling" (Rs. 800, 45 mins).
3. **Staff Selection:** Customer chooses "Daksh Patel (Master Stylist)" or selects "Any Available".
4. **Calendar Date:** Customer picks an upcoming Friday.
5. **Slot Generation:** Frontend fetches `GET /api/appointments/slots`. The slot engine calculates all available 45-minute windows taking into account Daksh's shifts, lunch breaks, and existing bookings.
6. **Booking Confirmation:** Customer selects 11:15 AM, enters special notes, and clicks "Confirm Booking".
7. **Atomic Creation:**
 - Backend verifies slot validity, locks the unique index entry, snapshots `priceAtBooking: 800` and `durationAtBooking: 45`.
 - Generates an in-app notification.
 - Dispatches a Brevo confirmation email containing an attached `invite.ics` calendar file.
 - Redirects customer to confirmation screen displaying a 1-click Google Calendar sync button.

3.1.2 Journey 2: Specialist Service Execution

1. Specialist logs into `/staff` portal and views today's chronological agenda.
2. Customer arrives on premises at 11:10 AM. Specialist clicks **"Check-In"**. Status transitions from `scheduled` to `confirmed`.
3. Service completes at 11:55 AM. Specialist clicks **"Complete Appointment"**.
4. Status transitions to `completed`. The business's total completed appointment counter increments.
5. Brevo automatically dispatches a "Session Completed" email to the customer with an invitation to leave feedback.

3.1.3 Journey 3: Verified Review Scorecard

1. Customer logs in and opens `/my-appointments` or clicks the link in their post-session email.
2. A **"Write Review"** button is active exclusively for this completed appointment.
3. Customer awards 5 stars and writes feedback.
4. Backend verifies that:
 - Appointment status is `'completed'`.
 - Requesting user matches `appointment.customerId`.
 - No review exists yet for this `appointmentId`.
5. Review is saved. The business's `averageRating` and `totalReviews` are recalculated, and the review appears dynamically on the public storefront.

3.1.4 Journey 4: Front-Desk Walk-In (POS Mode)

1. An unregistered walk-in customer walks into a medical clinic.
2. Receptionist opens `/admin` dashboard and clicks **"+ New Walk-In"**.
3. Receptionist selects the service, assigns an available doctor, enters the customer's name and phone number.
4. Backend creates an appointment with `isWalkIn: true` and `customerId: null`.
5. The doctor's schedule immediately blocks out the slot, preventing online clients from double-booking.

3.2 Complete REST API Specification

Authentication (/api/auth)

- POST /api/auth/register : Registers a new account.
- POST /api/auth/login : Authenticates credentials.
- GET /api/auth/me : Retrieves the profile of the current user.
- POST /api/auth/refresh-token : Issues a new access token.

Appointments (/api/appointments)

- GET /api/appointments/slots : Computes available time slots.
- POST /api/appointments : Creates a new appointment with atomic lock.
- POST /api/appointments/walk-in : Creates a walk-in appointment.
- PATCH /api/appointments/:id/reschedule : Reschedules an appointment.
- PATCH /api/appointments/:id/cancel : Cancels an appointment.

Businesses & Discovery (/api/business)

- GET /api/business/:slug : Fetches public storefront.
- PUT /api/business/settings : Updates operating hours and policies.
- GET /api/search?q=... : ReDoS-safe search.

Services (/api/services)

- GET /api/services/business/:businessId : Fetches all active services.
- POST /api/services : Creates a new catalog service.

Reviews & Analytics

- POST /api/reviews : Submits a star rating and comment.
- GET /api/analytics/overview : Revenue totals and booking counts.

Chapter 4

System Design

4.1 End-to-End System Architecture

Client Tier [Frontend SPA - React 18 + Vite]

- Neo-Bauhaus High-Contrast UI
- React Router DOM v7
- AuthContext & Auto-Refresh Interceptor
- Customer, Admin, and Staff Views

Gateway Tier [API Gateway - Express.js on Node.js]

- Express Engine with CORS Whitelist + Helmet + Rate Limiter
- JWT Verify + Role Guards + IDOR Prevention
- ReDoS-Safe Regex Sanitizer

Business Logic [Controllers & Computation Engines]

- Dynamic Slot Engine & Buffer Math
- Appointment Lifecycle Controller
- Revenue & Retention Aggregation
- OAuth 2.0 & Token Issuer

Storage & Cloud Tier [Data & External Providers]

- MongoDB Atlas Database (Partial Unique Index, Aggregation)
- Brevo v3 Transactional Email REST API
- Cloudinary Cloud Media CDN
- RFC-5545 iCal & Google Calendar Sync

4.2 Database Schema & Data Model Deep Dive

The database architecture is designed with strict multi-tenancy. Every business entity maintains independent data while sharing a unified MongoDB Atlas cloud database.

4.2.1 User Model

- name, email, password
- role: Enum (customer, admin, staff)
- businessId: Nullable reference to Business
- notificationPreferences and googleCalendarSync configurations

4.2.2 Business Model

- name, slug, category, adminId
- workingHours: Weekly operational schedule
- bookingSettings: Custom business policies (e.g., minAdvanceTime)

4.2.3 Service & Staff Models

- **Service:** duration, price, bufferTime, staffIds
- **Staff:** specialization, workingHours (overrides business hours), unavailableDates

4.2.4 Appointment Model

- Immutable Fields: priceAtBooking, durationAtBooking
- Timestamps: appointmentDate, startTime, endTime, blockedUntil
- status: Enum (scheduled, confirmed, completed, cancelled, no-show)

4.2.5 Crucial Unique Concurrency Index

```
appointmentSchema.index(  
  { businessId: 1, staffId: 1, appointmentDate: 1, startTime: 1 },  
  {  
    unique: true,  
    partialFilterExpression: { status: { $in: ["scheduled"] } },  
  }  
);
```

Chapter 5

Core Logic & Mechanics

5.1 How the Database Works with the Backend

The interaction between the Express backend and MongoDB Atlas is architected around **safe connection pooling, atomic writes, defense against race conditions, and immutable historical records.**

5.1.1 Connection Pooling & Event Lifecycle

Mongoose initializes a managed TCP connection pool:

- `maxPoolSize: 10`: Limits concurrent sockets to avoid exhausting MongoDB Atlas M0/M10 thread thresholds.
- `serverSelectionTimeoutMS: 5000`: Fails quickly during network partitioning.
- `socketTimeoutMS: 45000`: Prevents hanging sockets from blocking Express worker threads.

5.1.2 The Concurrency Problem & The Hardware-Enforced Solution

Slotify solves this via **Database-Level Partial Unique Constraints**:

- The partial filter expression `{ status: { $in: [ "scheduled" ] } }` ensures that only active scheduled bookings occupy the unique index bucket.
- When multiple users hit the server at the exact same millisecond, MongoDB's internal B-tree lock allows only one write to commit.
- The second write throws an atomic database exception: `MongoServerError: E11000 duplicate key error.`
- The centralized error handler inspects the error and rejects the subsequent requests with a 409 `Conflict` response.

5.1.3 Immutable Snapshot Pricing

To guarantee financial and historical accuracy:

- When an appointment is created, it snapshots `priceAtBooking` and `durationAtBooking`.
- Analytics aggregation pipelines group and sum `$priceAtBooking`. Audit trails, receipts, and business payouts remain historically true regardless of future rate card edits.

5.2 Dynamic Slot Engine & Concurrency Control

The availability engine runs in real-time when clients view the booking calendar.

5.2.1 Slot States Explained

1. **available**: Open for booking.
2. **booked**: An active appointment occupies this time window.
3. **too-soon**: Slot start time violates `minAdvanceTime`.
4. **too-far**: Date exceeds `maxAdvanceTime`.
5. **past**: The time has already passed.

5.2.2 Dynamic Slot Calculation Algorithm

1. Load working hours for `targetDate` and `timezone`.
2. If date is a holiday or business is closed, return empty set.
3. For each eligible staff member:
 - a. Check staff leave records and blackout dates.
 - b. Build candidate intervals from shift boundaries.
 - c. Exclude intervals overlapping breaks.
 - d. For each interval, calculate state (`booked`, `available`, `too-soon`, etc.).
4. Merge state results, sort, and return deduplicated slots.

5.3 Email Engine, iCalendar & Notification Systems

- **Brevo Transactional Email Engine**: Dispatches multi-role HTML emails for confirmations, reminders, cancellations, and reviews.
- **RFC-5545 iCalendar (.ics)**: Automatic generation of `.ics` files containing `METHOD:REQUEST` to sync with Apple/Outlook calendars.

- **Google Calendar 1-Click:** Direct deep links generated on the frontend for instant Google Calendar integration.

Chapter 6

Implementation and Configuration

6.1 Backend File-by-File Technical Breakdown

- **server.js:** Entrypoint for the backend daemon. Connects to MongoDB, configures CORS, initializes Passport.
- **Controllers:** Contains `authController.js`, `appointmentController.js`, `analyticsController.js`, etc.
- **Middleware:** `auth.js` (JWT verification, RBAC), `errorHandler.js` (Centralized errors), `validateBusinessAccess.js` (IDOR prevention).
- **Models:** Mongoose schemas (`User.js`, `Business.js`, `Appointment.js`, etc.).
- **Utils:** `availabilityCalculator.js` (Dynamic slot algorithm), `emailService.js` (Brevo integration).

6.2 Frontend Architecture & State Management

Constructed using **React 18.2** and bundled via **Vite 7**. Adheres to a strict **Neo-Bauhaus design system**.

6.2.1 Authentication & Token Refresh Flow

1. **App Mount:** Validates `accessToken` via `GET /api/auth/me`.
2. **Axios Interceptor:** Attaches Bearer tokens to outgoing requests.
3. **Transparent 401 Refresh:** If a request fails with 401, issues a silent `POST /api/auth/refresh-token` with HTTP-only cookies and replays the original request.

6.3 Security, Performance & Error Handling Matrix

- **ReDoS Protection:** Search inputs pass through regex sanitization neutralizing catastrophic backtracking.
- **Strict CORS Whitelist:** Production requests are verified against authorized origins.
- **IDOR Prevention:** Cross-tenant data leakage is prevented via business ID validation.
- **Password Protection:** Bcrypt salted hashing with 10 rounds.

6.4 Local Setup, Environment Variables & Deployment

Backend Configuration (.env):

PORT=5000

MONGODB_URI=...

JWT_SECRET=...

BREVO_API_KEY=...

CLOUDINARY_API_KEY=...

Deployment:

- Frontend: Vercel (SPA rewrite rules)
- Backend: Render (Node web service)
- Database: MongoDB Atlas (Cloud M0/M10 replica set)

Chapter 7

Conclusion

The Slotify platform successfully addresses the core challenges in modern appointment scheduling. By implementing a hardware-enforced concurrency lock at the database layer, we have completely eliminated the risk of double bookings. The immutable snapshot pricing architecture ensures that all financial and historical records remain accurate and immune to retroactive changes. Furthermore, the two-way calendar synchronization and verified review systems greatly improve the overall customer experience, while providing business owners with reliable management tools.

Slotify stands as an enterprise-grade, multi-tenant solution that seamlessly balances complex backend logic with an intuitive Neo-Bauhaus user interface.

7.1 Academic Citation & Institutional Affiliations

Slotify was conceptualized, designed, and engineered as the **Final Year Major Capstone Project** by project group **CSE_182** at **Parul University**.

7.1.1 Research Publication

- **Paper Title:** *Multi-Tenant Appointment Booking & Real-Time Scheduling Architecture with Concurrency Lock & Snapshot Pricing*
- **Team / Group Code:** **CSE_182**
- **Affiliation:** Department of Computer Science & Engineering, Faculty of Engineering & Technology (PIET), **Parul University**, Vadodara, Gujarat, India.
- **Academic Batch:** 2022 – 2026 — **Program:** Bachelor of Technology (B.Tech) in Computer Science & Engineering.

7.1.2 Project Team & Authors

- **Aksh Thakkar** — Founder, Product Vision & Full-Stack System Architecture

- **Om Maurya** — Frontend Lead & UI/UX Specialist
- **Daksh Patel** — Backend & Systems Engineer
- **Jainil Patel** — Database & Concurrency Engineer

Chapter 8

Future Work

While Slotify provides a comprehensive solution for appointment scheduling, several enhancements could further optimize the platform for large-scale enterprise use. The proposed future extensions are:

- **Payment Gateway Integration:** Integration with platforms like Stripe or Razorpay to collect upfront deposits and cancellation fees automatically during the booking flow.
- **AI-Powered Booking Assistant:** A natural language conversational bot that allows clients to discover services, check availability, and reserve slots via chat.
- **Advanced Analytics & Machine Learning:** Implementing predictive models to forecast peak business hours, suggest dynamic pricing adjustments, and optimize staff schedules based on historical utilization data.
- **Offline Notifications via SMS/WhatsApp:** Integration with communications APIs like Twilio to deliver real-time alerts and reminders to clients without requiring internet access.
- **Mobile Application Extensions:** Developing native iOS and Android companion apps using React Native to offer advanced features like location-based check-ins and persistent push notifications.

References

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.